

Take Home Assignment

Senior ML Engineer

(LLM & Gen AI Specialist)

Technical Assessment

September 2026

Reasoning-Trace Dataset Generation

Objective and Success Criteria

This take-home exercise is designed to evaluate your ability to curate, filter, and structure high-quality synthetic data for supervised fine-tuning (SFT). Specifically, you will design a local pipeline that transforms raw biology Q&A pairs from our private Hugging Face repository into high-quality reasoning-trace datasets (similar to DeepSeek-R1 or o1 styles).

Please note the following expectations:

- **Deliver a functional prototype:** The solution does not need to be production-ready or fully optimized. The priority is a functional script or pipeline that we can run seamlessly.
- **Time box:** We expect you to spend **around 4 hours (excluding inference time)** on this exercise.
- **Local hardware constraints:** Your solution must run locally on a standard consumer laptop (e.g., Apple Silicon Mac with up to 16GB RAM) utilizing local, open-source models (e.g., 3B to 8B parameters).
- **AI Assistants:** You are allowed to use coding assistants (Copilot, Cursor, etc.).
- **Panel presentation:** If successful, you will be invited to present your solution to a panel, where we will discuss your filtering logic, choice of prompts, and technical trade-offs.

The goal is to assess your scientific methodology, pragmatism, critical thinking and technical judgment.

Expected behavior

You are expected to build a prototype data engineering pipeline in Python. The application should take our small collection of Q&A items, leverage a local open-source LLM to generate a structured reasoning chain (thinking process) for each, apply an automated filtering/validation gate, and output a clean SFT-ready dataset.

The application must:

1. Authenticate and pull the pre-built biology/pharmacogenomics Q&A dataset from our private Hugging Face repository.
2. Stream or loop through the Q&A pairs, passing the raw `question` to a local 3B–8B parameter model.
3. Use prompting techniques to force the model to output a structured reasoning trace (e.g., inside `<think>` tags) *before* arriving at the final response.
4. Programmatically validate the output: Check that the structure is correct and that the final response accurately aligns with the ground-truth answer from the HF sample (written into `hf_sample.answer` in the output).

The pipeline must output a clean `.jsonl` dataset following this structure:

JSON

None

```
{
  "question_id": 0,
  "completion_id": 0,
  "hf_sample": {
    "question": "At 95.0 nM concentration, which of these two drugs...?",
    "answer": "B",
    "options": {"A": "Hydrocortisone", "B": "CCT 018159"}
  },
  "completion": {
    "reasoning": "First, compare mechanism and expected hallmark shifts...",
    "answer": "B"
  },
  "prompting": {
    "full_prompt": "[exact prompt sent to the local model]"
  }
}
```

Your pipeline must wrap input HF into the object above. Store the inner `<think>` text in `completion.reasoning` (without the tags), and store the choice key (A/B) in `completion.answer`.

Technical Requirements

- **Portability:** The prototype must be provided as a GitHub repository that can be cloned and run end-to-end on another laptop without manual environmental debugging
- **HF integration:** Read a Hugging Face token dynamically from the environment (`HF_TOKEN`) to pull from the private space.
- **Documentation:** A `README.md` file detailing setup instructions, how to execute the pipeline, an overview of your design decisions, and an evaluation of your filtering logic.
- **Language and stack:** Python. You can use any lightweight local execution framework (e.g., `vllm`).
- **Hardware baseline:** Must execute efficiently on an Apple Silicon Mac laptop. Model sizes should be capped appropriately (e.g., ~3B to 8B models).
- **Robust parsing and filtering:** Your script must include specific heuristics to handle cases where the local model hallucinates, fails to use the formatting tags, or explicitly contradicts the dataset's ground truth.
- **Architecture and AI justification:** The README must explicitly describe the pros, cons, and performance trade-offs of using your selected local model size for reasoning generation.

Prerequisites

An M-series Mac and Docker Desktop 4.62 or later you can test either backend/model pairing:

Option A:

```
docker model install-runner --backend vllm
```

```
docker model pull hf.co/Qwen/Qwen2.5-3B-Instruct
```

```
docker model run hf.co/Qwen/Qwen2.5-3B-Instruct
```

Option B (recommended on Apple Silicon):

```
docker model install-runner --backend llama.cpp
```

```
docker model pull hf.co/Qwen/Qwen3-4B-GGUF
```

```
docker model run hf.co/Qwen/Qwen3-4B-GGUF
```

Available Data

Your repository should build upon or wrap an interface similar to the following setup:

Python

```
None
import os
from typing import List, Dict
from datasets import load_dataset

# Configuration for private Hugging Face repository
PRIVATE_REPO = "owkin/technical_test"

def load_private_dataset() -> List[Dict]:
    """Loads the pre-built Q&A dataset from Hugging Face."""
    token = os.getenv("HF_TOKEN")
    if not token:
        raise ValueError("Please set the HF_TOKEN environment variable.")

    # Expected features: 'question', 'answer', 'options'
    dataset = load_dataset(PRIVATE_REPO, token=token, split="train")

    # Return a sample subset to keep the exercise fast
    return list(dataset.select(range(min(50, len(dataset)))))

def generate_reasoning_traces(qa_samples: List[Dict]) -> List[Dict]:
    """
    TODO: Candidate implementation
    1. Loop through samples and present the 'question' to a local small LLM.
    2. Prompt the model to generate a structured <think> trace and final answer.
    3. Verify the parsed final answer against sample["answer"] (ground truth).
    4. Keep only valid rows; wrap each into the output schema (hf_sample + completion + prompting).
    """

if __name__ == "__main__":
    print("Fetching private Q&A dataset...")
    raw_qa = load_private_dataset()

    print(f"Loaded {len(raw_qa)} pairs. Running reasoning generation...")
    sft_dataset = generate_reasoning_traces(raw_qa)
    print(f"Generated {len(sft_dataset)} high-quality training pairs.")
```

